5OD21 – Optimisation Discrète
Projet : Course d'avions

Mohamed Benloughmari Fatimetou Narjah

1 Problème et graphe

On dispose de n aérodromes $V = \{1, \dots, n\}$, d'un départ d, d'une arrivée f, de K régions (on note V_k les aérodromes de la région k), d'un nombre minimal d'aérodromes A_{min} et d'un rayon d'action R. On cherche un chemin élémentaire de d à f, de longueur totale minimale, qui visite au moins A_{min} aérodromes (départ et arrivée compris), passe par chaque région et n'effectue que des sauts de longueur $d_{ij} \leq R$. C'est une variante du TSP orienté où l'on choisit les sommets visités et où l'on cherche un chemin $d \to f$ plutôt qu'un circuit.

On travaille sur le graphe orienté $G = (V, A)$ avec

$$A = \{(i,j) \in V^2 : i \neq j,\ d_{ij} \leq R,\ j \neq d,\ i \neq f\}.$$

La contrainte de rayon est ainsi prise en compte directement dans le graphe, et aucun arc n'entre en d ni ne sort de f. On note $\delta^+(k)$ (resp. $\delta^-(k)$) les arcs sortant de (resp. entrant dans) k.

2 Formulations

Variables et contraintes communes. $x_{ij} \in \{0,1\}$ vaut 1 si l'avion vole directement de i à j ; $y_i \in \{0,1\}$ vaut 1 si l'aérodrome i est visité.

$$\begin{aligned}
\min\ & \sum_{(i,j)\in A} d_{ij}\, x_{ij} \\
\text{s.c. } & \textstyle\sum_{\delta^+(d)} x_{dj} = 1, \qquad \sum_{\delta^-(f)} x_{if} = 1, \qquad y_d = y_f = 1 && (1)\\
& \textstyle\sum_{\delta^-(k)} x_{ik} = y_k, \qquad \sum_{\delta^+(k)} x_{kj} = y_k \qquad \forall k \in V \setminus \{d,f\} && (2)\\
& \textstyle\sum_{i\in V} y_i \geq A_{min} && (3)\\
& \textstyle\sum_{i\in V_k} y_i \geq 1 \qquad \forall k \in \{1,\dots,K\} && (4)
\end{aligned}$$

Les contraintes (1)–(2) imposent que les arcs choisis forment un chemin de d à f, auquel peuvent s'ajouter des circuits disjoints (sous-tours). Ces sous-tours permettent de « visiter » des aérodromes à bas coût pour satisfaire (3)–(4), il faut donc les éliminer. Les deux formulations diffèrent uniquement sur ce point.

Formulation 1 : MTZ (nombre polynomial de contraintes). On ajoute une variable continue $u_i \in [1, n]$ représentant la position de i dans le chemin :

$$u_d = 1, \qquad u_j \geq u_i + 1 - n\,(1 - x_{ij}) \quad \forall (i,j) \in A. \tag{5}$$

Si $x_{ij} = 1$ on obtient $u_j \geq u_i + 1$; en sommant le long d'un circuit $i_1 \to \cdots \to i_p \to i_1$ on aurait $0 \geq p$, ce qui est impossible. Si $x_{ij} = 0$, la contrainte est toujours satisfaite. Le modèle a $|A| + 2n$ variables et $O(|A|)$ contraintes. En revanche, sa relaxation continue est faible : pour x_{ij} fractionnaire, le terme $n(1 - x_{ij})$ rend (5) presque inactive.

Formulation 2 : coupes de connexité (nombre exponentiel de contraintes). On remplace (5) par

$$\sum_{\substack{(i,j)\in A \\ i\notin S,\ j\in S}} x_{ij} \;\geq\; y_m \qquad \forall S \subseteq V \setminus \{d\},\ \forall m \in S. \tag{6}$$

Si m est visité, tout ensemble S contenant m mais pas d doit recevoir un arc : m est donc atteignable depuis d. Un sous-tour C viole (6) pour $S = C$. C'est l'analogue des contraintes d'élimination de sous-tours du TSP, avec y_m au second membre car les sommets ne sont pas tous visités. Grâce à (2), elle est équivalente à la forme GSEC $\sum_{i,j\in S} x_{ij} \leq \sum_{i\in S} y_i - y_m$. Le nombre de contraintes est de l'ordre de $n\,2^{n-1}$: on les ajoute uniquement lorsqu'elles sont violées.

Problème de séparation. Étant donné (x^*, y^*) satisfaisant (1)–(4), trouver $S \subseteq V \setminus \{d\}$ et $m \in S$ tels que $\sum_{i\notin S,\, j\in S} x^*_{ij} < y^*_m$, ou montrer qu'il n'en existe pas.

- *Cas fractionnaire.* En prenant x^* comme capacités, on a pour m fixé $\min_{S\ni m,\ d\notin S} \sum_{i\notin S,\, j\in S} x^*_{ij}$ égal à la coupe minimale $d \to m$, donc au flot maximal de d à m (théorème flot max / coupe min). Pour chaque m tel que $y^*_m > 0$, on calcule ce flot (Edmonds–Karp). S'il est strictement inférieur à y^*_m, l'ensemble S des sommets non atteignables depuis d dans le graphe résiduel fournit une coupe violée. La séparation est donc exacte et polynomiale (n calculs de flot).
- *Cas entier.* Les arcs choisis forment un chemin et des circuits. On suit le chemin depuis d ; chaque circuit C non atteint donne les coupes violées $(S = C, m)$ pour tout $m \in C$.

3 Mise en œuvre

Les modèles sont écrits en Julia avec JuMP et résolus avec SCIP, avec une limite de 600 s par résolution. Pour la formulation 2, SCIP.jl ne prend pas en charge les *lazy callbacks* de JuMP (le code les utilise si l'on choisit Gurobi). Nous utilisons donc une génération de contraintes itérative : on résout le PLNE, on sépare les sous-tours de la solution entière, on ajoute les coupes, puis on recommence jusqu'à obtenir une solution sans sous-tour. Celle-ci est optimale, puisqu'elle est optimale pour un relâché du modèle complet et réalisable pour ce dernier. Le temps et le nombre de nœuds indiqués sont cumulés sur les itérations.

La borne de relaxation continue à la racine (z_{LP}) est obtenue directement pour MTZ. Pour la formulation 2, on applique un algorithme de plans coupants sur le PL relâché, avec la séparation fractionnaire exacte, jusqu'à ce qu'aucune coupe ne soit violée. Les coupes ainsi trouvées sont conservées dans le PLNE de départ, et le temps de cette phase est inclus dans le temps total et dans la limite de 600 s. Chaque solution est vérifiée indépendamment (chemin $d \to f$ élémentaire, au moins A_{min} sommets, sauts $\leq R$, toutes les régions visitées). Sur l'exemple du sujet (6 aérodromes), on retrouve l'optimum 12 avec le parcours $1 \to 3 \to 2 \to 5$. Pour `aerodrome_90_1`, qui déclare $K = 6$ mais attribue la région 7 à 5 aérodromes, nous respectons K et considérons ces aérodromes comme hors région.

4 Résultats

Le tableau 1 donne, pour chaque instance, la meilleure valeur connue z^* (optimale sauf mention), la borne de relaxation continue à la racine z_{LP}, le gap racine $(z^* - z_{LP})/z^*$, le temps total et le nombre de nœuds du Branch-and-Cut. Pour la formulation 2, le nombre de nœuds est cumulé sur les itérations.

Lorsque MTZ atteint la limite de temps, la meilleure solution trouvée et la borne inférieure finale sont, pour (80_1, 90_1, 100_1, 100_2, 120_1, 150_1) : (385 ; 367.8), (689 ; 545.0), (690 ; 639.1), (611 ; 559.3), (481 ; 388.7) et (456 ; 437.1).

			MTZ				Coupes de connexité			
Instance	$\|A\|$	z^*	z_{LP}	gap %	temps (s)	nœuds	z_{LP}	gap %	temps (s)	nœuds
0	18	12	8.0	33.3	0.0	1	10.0	16.7	0.3	1
20_1	288	73	65.2	10.8	0.0	1	71.0	2.7	0.1	1
20_2	286	91	63.0	30.8	0.7	67	74.5	18.1	0.0	1
20_3	113	82	64.0	22.0	0.0	1	72.5	11.6	0.0	1
20_4	226	96	78.0	18.8	0.1	10	86.5	9.9	0.0	1
30_1	156	115	84.1	26.9	0.6	623	110.5	3.9	0.9	2
40_1	417	124	114.2	7.9	0.0	1	124.0	0.0	0.6	1
50_1	441	148	137.0	7.4	0.3	1	148.0	0.0	1.6	1
70_1	757	288	231.1	19.8	7.7	1754	278.8	3.2	10.3	1
80_1	608	385	221.3	42.5	>600	491822	385.0	0.0	27.4	1
90_1	1287	605	409.8	32.3	>600	258278	604.3	0.1	62.0	2
100_1	957	665	429.5	35.4	>600	186610	658.7	0.9	54.4	2
100_2	1205	580	410.9	29.1	>600	208444	577.9	0.4	61.0	2
120_1	5473	481†	248.6	48.3	>600	67337	410.0	14.8	>600	0
150_1	1098	448	352.2	21.4	>600	243733	446.9	0.2	294.2	1

TABLE 1 – Résultats avec SCIP (limite 600 s). † meilleure solution trouvée, optimalité non prouvée : on sait seulement que $z^* \in [410.0,\ 481]$.

5 Analyse

Qualité des relaxations. Sur toutes les instances, la borne racine de la formulation par coupes est meilleure que celle de MTZ. Le gap moyen passe d'environ 26 % (MTZ) à environ 5.5 % (coupes), et il est inférieur à 1 % sur toutes les instances résolues avec $n \geq 80$. Le gap de MTZ, au contraire, ne diminue pas quand n augmente : il dépasse 30 % sur la plupart des grandes instances. C'est le résultat attendu : avec x_{ij} fractionnaire, la contrainte $u_j \geq u_i + 1 - n(1 - x_{ij})$ est presque toujours satisfaite, alors que les coupes de connexité imposent, même en fractionnaire, qu'un flot y_m puisse être envoyé de d vers chaque sommet m. On retrouve la hiérarchie connue pour le TSP : la relaxation de la formulation exponentielle est plus forte que celle de MTZ.

Effort de résolution. Pour $n \leq 70$, les deux formulations résolvent les instances en quelques secondes. MTZ peut toutefois demander beaucoup de nœuds (1754 pour 70_1), alors que la formulation par coupes termine presque toujours à la racine (1 ou 2 nœuds). À partir de $n = 80$, MTZ ne prouve plus l'optimalité en 600 s malgré plusieurs centaines de milliers de nœuds. Sa meilleure solution peut rester loin de l'optimum : 689 contre 605 pour 90_1, soit 14 % de plus. La formulation par coupes résout ces instances en 30 s à 5 min. Le temps est alors presque entièrement consacré aux plans coupants de la racine (environ n calculs de flot maximal et un PL par itération) : c'est le prix d'une relaxation forte.

Limites. Le coût de la racine croît avec le nombre d'arcs. Sur 120_1 ($R = 90$, donc 5473 arcs, graphe presque complet), les plans coupants épuisent à eux seuls la limite de temps, et aucune solution entière n'est obtenue par cette formulation. Sa borne (410.0) reste néanmoins meilleure que la borne finale de MTZ après 600 s (388.7). Deux améliorations simples seraient de limiter le temps passé à la racine, ou d'utiliser un solveur qui accepte les *lazy callbacks* (Gurobi) pour séparer les coupes au cours du Branch-and-Cut plutôt que par résolutions successives. Le code prévoit déjà cette seconde option.

Conclusion. MTZ est simple à écrire et suffit pour les petites instances, mais sa relaxation faible le rend inutilisable au-delà de 80 aérodromes. La formulation exponentielle, associée à une séparation exacte par flot maximal, donne une borne quasiment égale à l'optimum et résout 14 des 15 instances à l'optimalité, contre 9 pour MTZ.